

Freight Rate Prediction Challenge

Final Report - Model Development, Validation, and Results

1. Executive Summary

This project builds a machine learning model to predict truckload freight rates (**posted_rate**) from load characteristics such as pickup/delivery location, distance, equipment type, weight, and date.

Item	Value
Training/development rows	48,000 (data/train-test.csv)
Validation rows to predict	12,000 (data/validation.csv)
Final model	HistGradientBoostingRegressor (tuned), log1p target
Final validation metrics (avg. of 3 time windows)	MAE 119.40 RMSE 627.96 R2 0.8260
Final deliverables	validation_predictions.csv, data/december_predictions.csv, scorer_results/candidate_december.png

2. Problem Statement

The task is to predict the freight rate (**posted_rate**) for each of the 12,000 loads in data/validation.csv, using a model trained on 48,000 historical loads with known rates. Predictions must be delivered as validation_predictions.csv with exactly the columns load_id,predicted_rate. A second, smaller task requires predicting rates for a fixed December route across all 31 days of the month, used to visually check how the model's predictions behave over time.

3. Dataset Overview

File	Rows	Purpose
data/train-test.csv	48,000	Labeled data (has posted_rate) for training/validation
data/validation.csv	12,000	Unlabeled loads to predict for final submission
data/december-chart-inputs.csv	31	One fixed lane, one row per December day

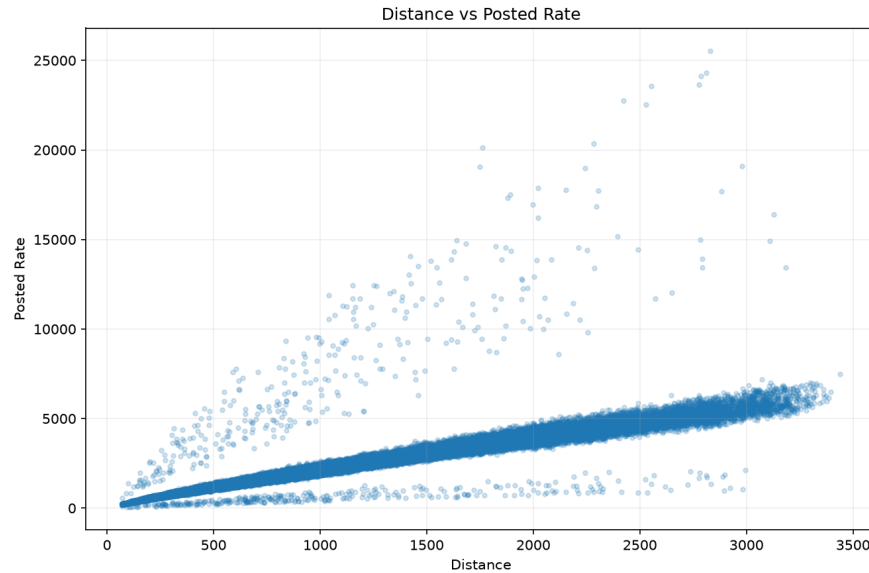
Key columns

- **pickup / delivery**: origin and destination city
- **pickup_lat/lon, delivery_lat/lon**: coordinates
- **distance**: shipment distance
- **equipment**: Dry Van, Reefer, or Flatbed
- **weight**: load weight
- **date**: shipment date
- **market_index, quote_signal**: market-related numeric signals
- **posted_rate**: the target variable (training data only)

4. Exploratory Data Analysis

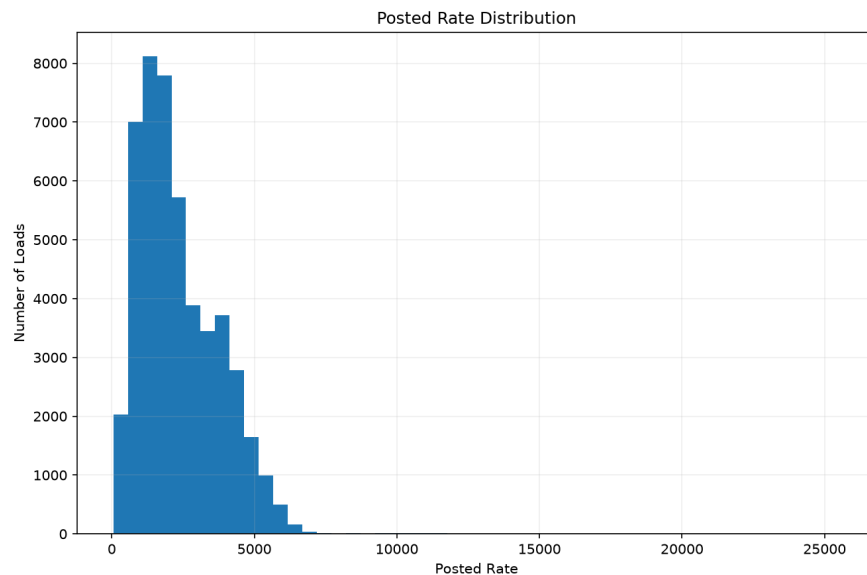
The charts below were generated directly from the training data (eda/generate_plots.py) using the same logic and findings from the original exploratory analysis (eda/explore_data.py).

Distance vs Posted Rate



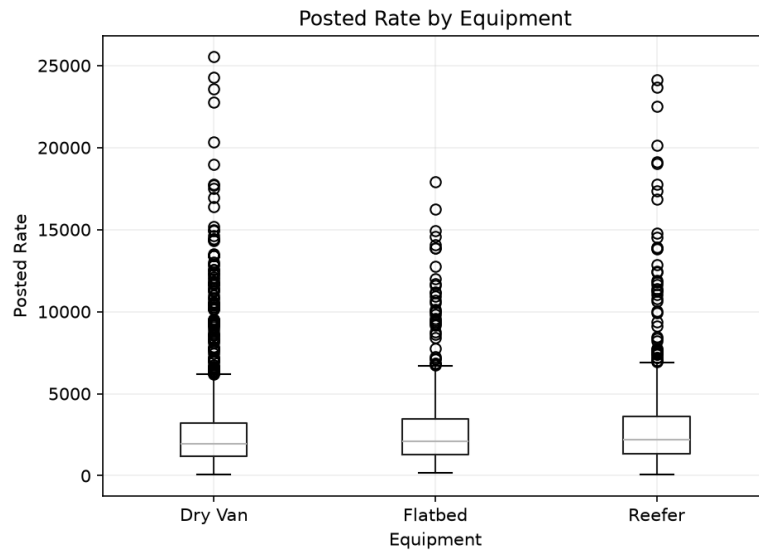
Distance shows a clear, strong positive relationship with posted_rate (correlation approximately 0.91) - the strongest individual relationship found during EDA, and the main driver of model accuracy.

Posted Rate Distribution



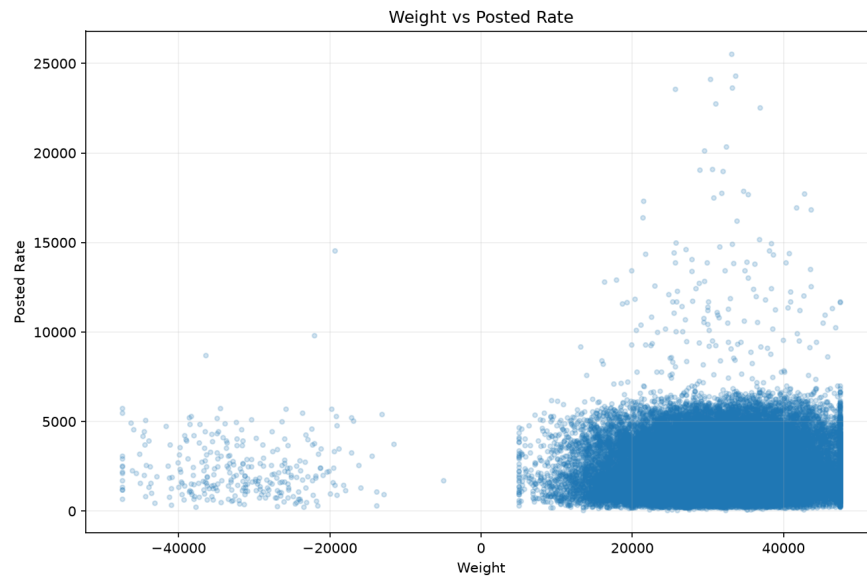
posted_rate is right-skewed: most loads sit at lower rates, with a long tail of rare, high-value loads. This motivated testing a $\log_{1p}(\text{posted_rate})$ target transform during modeling.

Posted Rate by Equipment



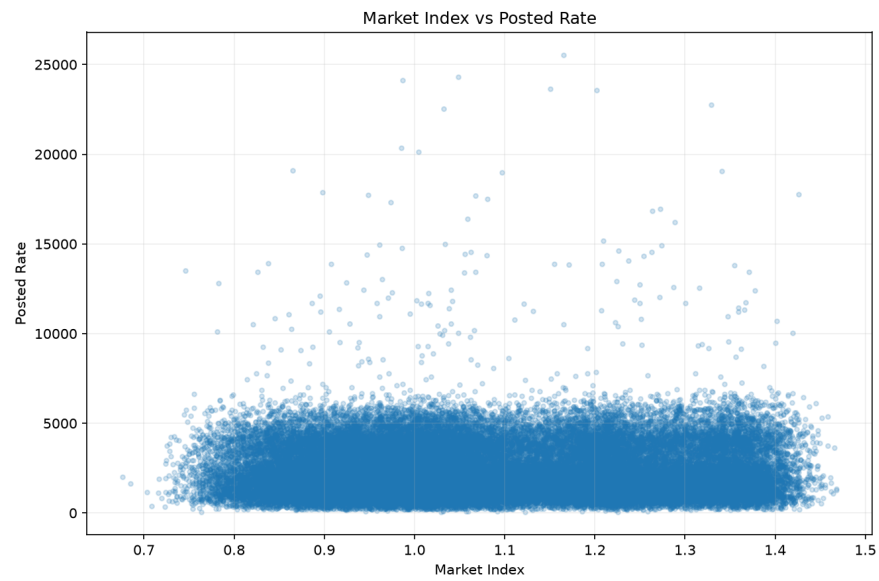
Rate distributions differ modestly by equipment type, with overlapping ranges across Dry Van, Reefer, and Flatbed. Equipment was retained as a feature.

Weight vs Posted Rate



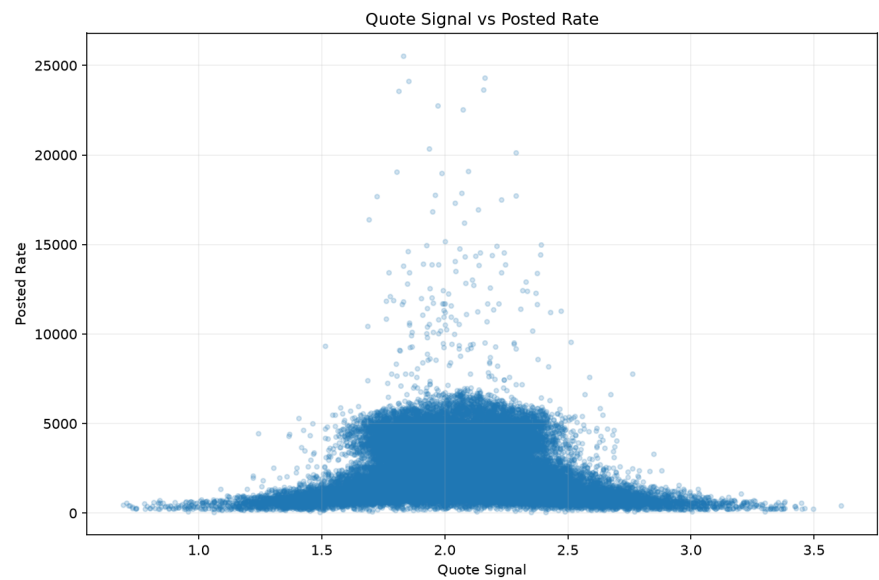
The direct relationship between *weight* and *posted_rate* is weak. This chart also makes a data-quality issue clearly visible: a cluster of negative weight values, which do not make physical sense and were treated as data-entry errors.

Market Index vs Posted Rate



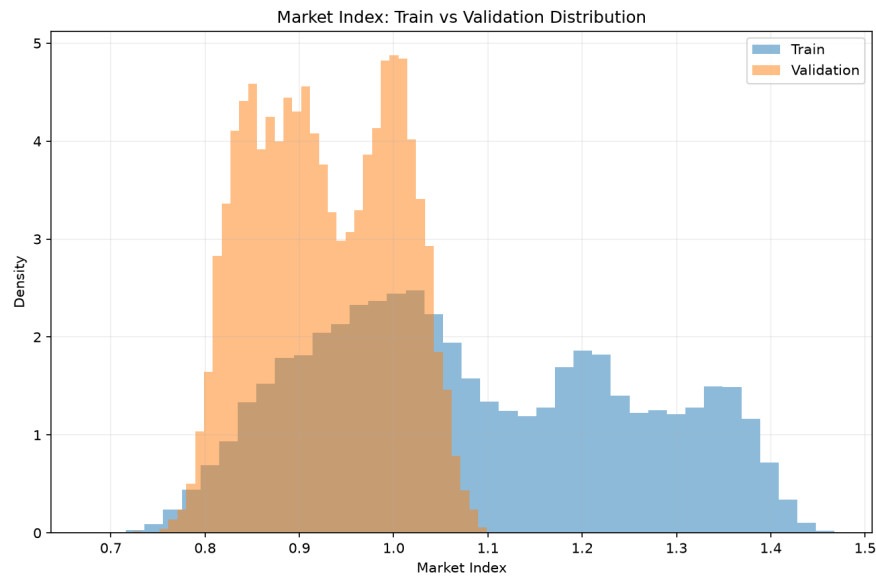
No strong direct linear relationship is visible between `market_index` and `posted_rate`, consistent with a low measured correlation. The feature was kept since tree-based models can still use non-linear signal.

Quote Signal vs Posted Rate



Similarly, `quote_signal` shows a weak direct linear relationship with `posted_rate` and was retained for the same reason.

Market Index: Train vs Validation Distribution



Validation's market_index values are noticeably lower than training's. This is a genuine distribution shift, not a data error, and was documented rather than corrected (see Section 5).

Key EDA findings

- Distance is the strongest individual predictor found in the data.
- posted_rate is right-skewed with a long high-rate tail.
- Equipment type is associated with modest differences in typical rate.
- Negative weight values were identified as a data-quality issue, not a real category.
- market_index shows a notable distribution shift between training and validation periods.
- Validation contains cities not present in training, so raw city names were not relied on as model features.

5. Data Quality and Preprocessing

All cleaning decisions were made using statistics computed from the training data only, then applied identically to validation and December data, to avoid leaking information from data the model should not have access to.

- No duplicate rows were found in the training or validation data.
- Missing weight values were filled using the training-data median.
- Negative weight values were converted using `abs(weight)`.
- A `weight_was_negative` flag column was created to preserve this information for the model.
- A `weight_was_missing` flag column was created for originally missing weight values.
- Missing `market_index` values were filled using the training-data median.
- A `market_index_was_missing` flag column was created.
- Date features (month, day of week, and a cyclical day-of-year encoding) were created.
- Raw pickup/delivery city names were dropped from model features, since validation contains cities never seen in training.
- Pickup/delivery coordinates (latitude/longitude) were retained, since they are well-defined for any city, seen or unseen.
- Raw input data files (`data/train-test.csv`, `data/validation.csv`, `data/december-chart-inputs.csv`) were never modified.

The negative-weight treatment (`abs(weight) + flag`) was not just a modeling assumption - it was experimentally compared against two alternatives (treating negatives as missing, and leaving them unchanged) using the same time-based validation windows described in Section 7. The `abs(weight)` approach produced the lowest, most consistent error across all three windows and was kept for the final model.

6. Feature Engineering

Feature engineering focused on turning cleaned columns into numeric signals a tree-based model can use directly, while keeping the feature set simple.

- **Equipment one-hot encoding:** Dry Van / Reefer / Flatbed converted to three binary columns.
- **delta_lat, delta_lon:** difference in latitude/longitude between pickup and delivery.
- **haversine_distance:** straight-line distance between pickup and delivery, computed purely from coordinates so it works for any city, including ones unseen in training.
- **log_distance:** $\log(1 + \text{distance})$, to reduce the influence of very long hauls.
- **route_directness:** actual distance divided by straight-line distance - a higher value means a more indirect route.
- **distance x equipment interactions:** lets each equipment type have its own rate-per-mile relationship rather than assuming one shared slope.
- **Calendar features:** month, day of week, and a sine/cosine pair representing day of year, so that December 31 and January 1 are treated as numerically close.

7. Validation Strategy

Freight rates are time-dependent, and the actual task is to predict a future period (validation.csv begins in November) using only past data (training data runs January-October). For this reason, a random train/test split was deliberately avoided, since it would let the model be evaluated on dates earlier than some of its own training data - not a fair test of forecasting ability.

Instead, a chronological split was used, and the evaluation was repeated across three separate time windows to confirm the result was stable rather than specific to one arbitrary cutoff:

Window	Training period	Validation period
Window 1	Jan 1 - Jun 30	Jul 1 - Aug 31
Window 2	Jan 1 - Jul 31	Aug 1 - Sep 30
Window 3	Jan 1 - Aug 31	Sep 1 - Oct 31

Measuring performance across all three windows (rather than a single split) gives a more reliable estimate of how the model will perform going forward: a model that only does well on one specific window could simply be lucky on that slice of data. In this project, MAE remained reasonably stable across all three windows, with the best-to-worst window differing by about \$13, which supports treating the final metrics as a realistic estimate of performance rather than an optimistic one. data/validation.csv itself was never used for training, tuning, or model selection at any point - it has no true rate to compare against and is reserved solely for the final submission.

8. Model Development

Models were developed through a sequence of controlled experiments, each changing exactly one variable and evaluated across the same three time windows. Full experiment code is in the experiments/ folder.

Step	Approach	MAE	RMSE	R2	Outcome
1	Random Forest (raw target)	186.29	686.16	0.7978	Baseline
2	Random Forest + log1p target	169.41	656.64	0.8149	Improved, kept
3	HistGradientBoosting + log1p	128.98	639.57	0.8244	Improved, kept
4	Multi-window stability check	-	-	-	Confirmed stable across windows
5	HGB hyperparameter tuning	119.40	627.96	0.8260	Final model
6	XGBoost (baseline config)	145.07	638.03	0.8203	Did not beat HGB
7	Feature experiments (interactions)	118.9-121.3	-	-	None improved consistently; dropped
8	Negative-weight treatment test	119.40 (best)	-	-	Confirmed current approach best
9	CatBoost (baseline config)	119.87	627.94	0.8260	Tied HGB, not better
10	Local HGB refinement	119.40 (best)	-	-	No nearby config improved consistently

Each step's improvement (or rejection) was judged on consistency across all three time windows, not on average performance alone - a change that helped one window while hurting another was not kept, even if its average looked favorable.

9. Final Model

The final model is a tuned **HistGradientBoostingRegressor**. It was selected because it consistently outperformed Random Forest, untuned XGBoost, and CatBoost across all three validation windows, and a further local search around its tuned parameters and several engineered features failed to find a consistent improvement - indicating this configuration is a stable, well-supported choice rather than a result of chance.

Parameter	Value
learning_rate	0.05
max_iter	300
max_leaf_nodes	15
max_depth	10
min_samples_leaf	50
l2_regularization	1.0

The model is trained on $\log_{1p}(\text{posted_rate})$ rather than the raw rate, since `posted_rate` is right-skewed; predictions are converted back to dollars using `expm1()`. This was tested and confirmed to improve accuracy over training on the raw target directly.

Final average metrics across the 3 validation windows

Metric	Value	Meaning
MAE	\$119.40	On a typical load, the prediction is off by about \$119
RMSE	\$627.96	Larger than MAE because a small number of large errors (rare high-rate loads) weigh heavily in this metric
R2	0.8260	The model explains approximately 82.6% of the variance in <code>posted_rate</code> on unseen, later dates

10. Error Analysis

Errors from the final model configuration were analyzed by group, pooling holdout predictions across all three validation windows.

Group	MAE
Distance 0-500 miles	\$41.32
Distance 2,500+ miles	\$233.52
Equipment: Dry Van	\$116.20
Equipment: Flatbed	\$119.84

Equipment: Reefer	\$126.24
Loads with posted_rate > \$6,000 (269 rows, 0.9% of data)	\$3,871.67
All other loads (posted_rate <= \$6,000)	\$83.80

Negative-weight rows and missing-weight rows were also checked specifically: neither group showed higher error than the rest of the data, so they are not a meaningful source of remaining error.

The clearest pattern is that error is concentrated in a small number of rare, high-rate, long-distance loads. This is a genuine limitation of the current model, not something that has been resolved - addressing it further would likely require a different modeling approach for that segment, which was outside the scope of this project.

11. Final Validation Predictions

The final model was retrained on all 48,000 rows of the training/development data (not just one time window), using the same tuned configuration, and used to predict every row of data/validation.csv.

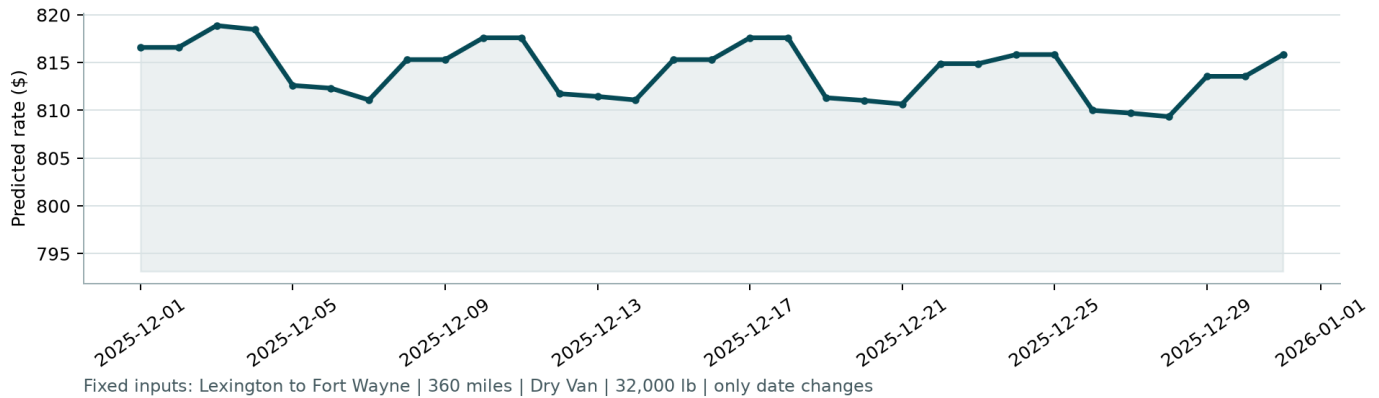
Check	Result
Output file	validation_predictions.csv
Row count	12,000
Unique load_id values	12,000
Missing predicted_rate values	0
Negative predicted_rate values	0
Columns	load_id, predicted_rate (exact match to required format)

12. Official December Prediction

The same final model was used to predict all 31 rows of data/december-chart-inputs.csv, a fixed Lexington to Fort Wayne route repeated once for each day of December 2025. All 31 predictions are positive and fall within a narrow, plausible range, since only the date changes row to row.

The chart below is the **official** chart, produced directly by the provided score.py after validating the required output format. score.py does not calculate model accuracy - it only validates that the submission files meet the required structure and generates this chart.

Candidate: December 2025 Predicted Load Rate



Official chart: `scorer_results/candidate_december.png`, generated by ``python score.py --predictions validation_predictions.csv --december-predictions data/december_predictions.csv``.

13. Limitations and Future Improvements

- High-rate, long-distance loads remain the model's weakest segment (Section 10) and account for a disproportionate share of total error.
- The December chart inputs require the fixed assessment format (one lane, one row per day), which limits how much can be inferred about general time-based behavior from that chart alone.
- A specialized approach for the high-rate tail (for example, a separate model or error-weighted training) remains untested and could be investigated further.

After the final model was selected, 4 further ideas were tested to see if it could still be improved: predicting rate-per-mile instead of the raw rate, extra coordinate-derived geometry features, historical average rate per route, and explicit long-haul/high-rate flags. All four were rejected using the same 3-window consistency rule used throughout this project - none improved results, and historical route averages made results notably worse (too noisy with only 64 distinct routes in the data). This is evidence the tuned model is close to the practical ceiling of what this feature set and dataset can support, rather than a model that was only lightly checked before finalizing. Full details are in `experiments/extras/`.

14. Reproducibility

Exact commands to reproduce the final pipeline and outputs:

```
python -m pip install -r requirements.txt
python preprocessing/run_preprocessing.py
python feature_engineering/run_feature_engineering.py
python modeling/train_final.py
python modeling/predict_december.py
python score.py --predictions validation_predictions.csv --december-predictions
data/december_predictions.csv
```

The repository also includes an automated test suite (`tests/`, run with `pytest tests/`) covering the preprocessing and feature engineering logic plus the final submission file formats, a GitHub Actions workflow that runs these tests on every push, and a small command-line demo (`modeling/predict_single_load.py`) that predicts a rate for any single load using the exact final model.

15. Conclusion

This project delivers a tuned HistGradientBoostingRegressor trained on cleaned, leak-free features, validated with a chronological, multi-window strategy appropriate for a forecasting task. Every modeling decision - target transform, negative-weight handling, feature choices, model family, and hyperparameters - was tested through controlled experiments and kept only when it improved results consistently, not just on average. The final model achieves an average MAE of \$119.40 and R2 of 0.8260 across three independent validation windows, with the main known limitation being reduced accuracy on rare, high-rate, long-distance loads. Final predictions for both validation.csv and the December chart inputs have been generated and validated against the official scorer.